A Compositional Mathematics of Optimization Fields

Probabilistic Categories, Optics, Free Plans, and Plugin Semantics for RAG Optimization

Architecture Research Thesis and Executable Sandbox

August 2026

Contents

Abstract

Optimization infrastructure is often built backwards. A system begins as a loop that enumerates parameter values, invokes an evaluator, writes a score, and selects a winner. As the domain grows, the loop accumulates exceptions: some candidates require rebuilding indexes, some only change query policy, some evaluations are stochastic, some failures must remain in the denominator, some parameters are coupled, some metrics are incomparable, and some changes are illegal because they violate security or deployment invariants. The result is usually an orchestration framework whose abstractions are operational accidents rather than mathematical structures.

This thesis develops an alternative foundation. The central object is an **optimization field**: a compositional family of parameterized systems, admissible interventions, stochastic observations, measurements, decision relations, and state transitions. The framework is motivated by retrieval-augmented generation, but its core is deliberately domain-neutral. RAG is grafted onto it through plugin signatures describing chunking, indexing, retrieval, answer generation, evaluation, and serving operations. The same core can host database tuning, prompt optimization, compiler exploration, model selection, or serving-policy experiments without importing those domains into the kernel.

The mathematical backbone combines several structures. A free symmetric monoidal category supplies a typed syntax for compositional plans. A parameterized-category construction separates system inputs from tunable parameters. Lawful lenses and related optics describe local interventions in large immutable specifications. Markov kernels and Kleisli composition describe stochastic evaluators and provider behavior. Couplings formalize paired evaluation and common random numbers. Measurements form structured observations rather than a universal scalar objective. Optimization campaigns are modeled coalgebraically as feedback processes that propose, evaluate, decide, and update while accumulating immutable evidence. Provenance is treated as part of semantics rather than as logging after the fact.

A key design principle is **interpretable syntax**. Plugins do not receive unrestricted control of the orchestration engine. They register typed primitive operations, schemas, laws, dependency declarations, and evaluators. The core constructs free plans from those generators. The same plan can then be interpreted into execution, static effect analysis, dependency closure, cost estimation, identity, diagrams, reference tests, or policy checks. This gives plugins extensibility without surrendering semantic control.

The thesis is accompanied by a self-contained Go sandbox named opfield. The implementation includes typed operation descriptors, a plugin registry, free sequential and monoidal plans, static plan analysis, an executor, deterministic content identities, lawful lenses, a finite-distribution monad, reproducible seed splitting, typed optimization spaces, paired campaign coordinates,

append-only resumable evidence, and a miniature RAG plugin. The demo compiles and runs without external dependencies. Its purpose is not to be production middleware. It is an executable specimen demonstrating that the proposed structures can remain small, understandable, and useful in ordinary Go.

The principal conclusion is that a reusable optimization architecture does not require a universal optimizer. It requires a small algebra of composition, intervention, observation, and evidence, together with plugin interfaces that preserve those semantics. The optimization strategy itself may vary widely. What composes is the field in which strategies act.

Preface

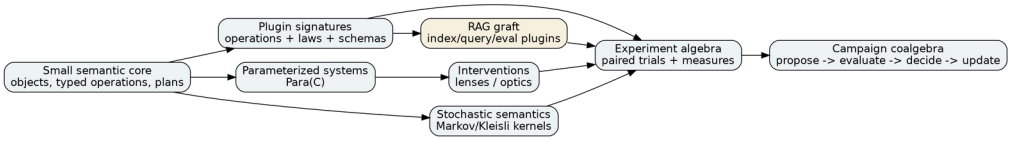
This work expands the optimization chapter of the larger RAG architecture study into a standalone mathematical and software design. The earlier work established that ragopt already has a sound practical nucleus: immutable candidates, exact paired cells, resumable run custody, comparison, ordered gates, and reviewable reports. It also established that the RAG domain itself must remain outside the generic optimization kernel. The remaining question is deeper: **what abstract structure should the optimization kernel represent so that real optimization domains can be attached without turning the core into a pile of callbacks?**

The answer developed here is intentionally conservative. Category theory is used where it gives concrete architectural leverage: typed composition, separation of syntax and interpretation, parameter composition, intervention laws, stochastic composition, and feedback. The implementation does not expose category-theory vocabulary to every caller. A Go developer sees operations, plans, plugins, spaces, trials, and campaigns. The formal model explains why those interfaces compose and what laws they must obey.

The reader should distinguish three levels throughout:

1. **Mathematical model.** This states ideal structures and laws.
2. **Software kernel.** This realizes a finite, explicit subset of those structures.
3. **Domain graft.** This maps concrete RAG systems onto the kernel through plugins and adapters.

The thesis does not claim that all optimization can be reduced to gradient descent, Bayesian optimization, or one generic search algorithm. It argues almost the opposite. Search strategy should be replaceable because the stable abstraction lies below it.



The proposed optimization backbone and its RAG graft.

Part I. The optimization field as a mathematical object

1. Why optimization needs a domain model

1.1 From loops to fields

A typical optimization program is written as:

```
for candidate in candidates:
   score = evaluate(candidate)
return argmax(score)
```

This is adequate only when candidates are already materialized, evaluation is deterministic, every candidate has the same cost, failures can be discarded, all metrics reduce to one scalar, and the system under test has no internal dependency structure. None of these conditions holds for serious RAG optimization.

Changing an RRF weight may require only a new query run. Changing a chunker requires rebuilding chunk records, representations, embeddings, indexes, and every downstream evaluation. Changing an embedding model may preserve lexical artifacts but invalidate vector material. Changing a provider may change not only quality but also cost, jurisdiction, disclosure policy, and stochastic behavior. A candidate can fail before producing a score, and that failure is itself evidence. Two candidates can be incomparable because one improves relevance while another reduces cost without crossing the allowable quality margin.

The primitive object must therefore contain more than a set of points and an objective function.

1.2 Definition of an optimization field

An optimization field is represented abstractly by the tuple

$$F = (S, I, W, K, M, \preceq, E),$$

where:

- S is a space of complete system specifications;
- I is a family of lawful interventions on those specifications;
- W is a workload or case space;
- K is the stochastic semantics of executing a specification on a case;
- M is a family of measurements over outcomes and traces;
- $\preceq$ is a decision relation or ordered gate program;
- E is an evidence algebra recording materialized observations and provenance.

An optimizer acts *inside* this field. Grid search, Bayesian optimization, evolutionary search, human proposal, or an LLM proposer can all generate interventions. They do not define the semantics of the field.

This separation solves an important architecture problem. The same RAG evaluation domain can be explored by different search strategies without rebuilding execution, evidence, or correctness

rules. Conversely, the same campaign engine can evaluate a database index plugin or prompt plugin without pretending their parameters have the same meaning.

1.3 Extensional and intensional optimization

Two system specifications can produce the same final score while differing materially in execution. One might use a fallback after a provider timeout. Another might disclose more text to a remote service. A third might return the same top-k but with a different latency distribution. Therefore the stochastic semantics must return a richer result:

$$K_s : W \to D(Y \times T),$$

where Y is the extensional outcome and T is an intensional trace.

Measurements can project either component:

$$m_j : Y \times T \to V_j.$$

This turns failure rate, provider calls, security events, cost, and cache behavior into first-class optimization evidence rather than side-channel telemetry.

1.4 Optimization as intervention rather than assignment

A parameter update should not be represented as an untyped map like {"chunk_size": 800}. The update occurs at a lawful focus in a complete specification. Let S be a specification and A a focused parameter. A lens provides

$$get : S \to A,\ put : S \times A \to S,$$

with the classical laws:

$$put(s, get(s)) = s,$$

$$get(put(s, a)) = a,$$

$$put(put(s, a), b) = put(s, b).$$

These laws are not mathematical decoration. They rule out surprising intervention interfaces. An optimizer that changes one declared focus should not silently reset unrelated fields or fail to expose the value it just set.

2. Composition before optimization

2.1 Systems as morphisms

Suppose a system stage consumes values of type A and produces values of type B. Denote it by a morphism

$$f : A \to B.$$

If a second stage is $g : B \to C$, then sequential composition gives

$$g \circ f : A \to C .$$

Independent stages compose monoidally:

$$f \otimes h : A \otimes X \to B \otimes Y .$$

RAG naturally uses both forms. Chunking then embedding is sequential. Lexical and vector channels are conceptually parallel. Fusion consumes the pair of rankings. Evaluation can run independent metrics over one outcome.

The first architectural claim is therefore simple: **the optimization core should understand the compositional shape of the system before it understands how to search its parameters.**

2.2 Why ordinary callbacks are too weak

A callback interface such as

```
type Step func(context.Context, any) (any, error)
```

can execute almost anything, but it exposes almost no semantic structure. The core cannot know which schemas connect, which effects occur, whether two steps can run independently, which dependencies a parameter invalidates, whether a step is deterministic, or how to compute a semantic identity without executing it.

A typed descriptor is stronger:

```
type Descriptor struct {
    ID           OperationID
    Inputs       Port
    Outputs      Port
    Effects      []Effect
    Dependencies []string
    Deterministic bool
    Cacheable    bool
    Cost         Cost
}
```

The implementation can still call arbitrary Go code at the primitive boundary, but composition is mediated by declared structure.

2.3 Symmetric monoidal structure

The useful wiring operations are:

- identity id_A;
- composition $g \circ f$;
- tensor $f \otimes g$;
- symmetry $\sigma_{A,B} : A \otimes B \to B \otimes A$;
- copying $\Delta_A : A \to A \otimes A$ when values are immutable and duplicable;
- dropping $!_A : A \to I$ when discarding is permitted.

The final two operations mean the implementation resembles a cartesian or gs-monoidal process syntax more than a purely linear monoidal category. That is appropriate for immutable software values. Crucially, the syntax does not imply that external effects may be copied. Copy duplicates a value reference or immutable envelope, not the effectful computation that produced it.

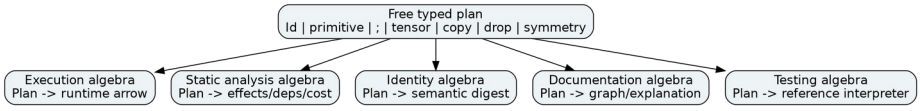
3. Free plans and interpreters

3.1 The free construction

Let Σ be a plugin-provided signature of primitive typed operations. Instead of allowing a plugin to implement arbitrary orchestration, the core constructs the **free typed process category** generated by Σ and the structural wiring operations.

A plan is syntax. It says which generators are connected and how. It does not yet say what those generators *mean operationally*.

This separation is the central plugin boundary of the architecture.



One free plan syntax admits many semantic interpreters.

3.2 Algebras over plan syntax

Given free syntax, an interpreter is an algebra that assigns meaning to primitives and preserves the structural operations. One algebra executes the plan. Another computes its effect set. Another accumulates cost. Another computes dependency closure. Another renders a graph. Another runs a deterministic reference model.

This yields a powerful invariant: all interpreters see the same topology because they fold the same plan tree.

The sandbox implements a simplified form with plan.Analyze and engine.Executor. A fuller production implementation should expose a generic fold interface. The concept is equivalent to interpreting a free algebra by a homomorphism into a target algebra.

3.3 Semantic identity

A plan can be given a content identity

$$id(p) = H(\text{canonical syntax}(p), \text{primitive semantic IDs}).$$

Execution placement, worker count, retry jitter, or timestamps do not belong in this identity unless they can alter the declared semantics. This separates semantic identity from execution identity.

The payoff is immediate: caches, experiment cells, audit logs, and release manifests can refer to a plan without serializing a closure or depending on process memory.

3.4 Static rejection

Because plans are typed and descriptors declare effects, the core can reject some invalid compositions before execution:

- schema mismatch;
- prohibited remote effect before authorization;
- non-cacheable operation in a claimed pure plan;
- candidate whose intervention closure omits an affected dependency;
- plan whose declared budget is exceeded by static lower bounds;
- plugin operation not available in the required capability set.

The more semantics that can be expressed declaratively, the less policy must be reconstructed from runtime logs.

4. Parameterized categories

4.1 Separating parameters from ordinary input

A tunable stage is not just $A \to B$. It is a family

$$f : P \times A \to B,$$

where P is the parameter object. A parameterized-category construction, commonly written $Para(C)$, makes such families compositional.

If

$$f : P \times A \to B$$

and

$$g : Q \times B \to C,$$

then their composite is parameterized by $P \times Q$:

$$g \circ f : (P \times Q) \times A \to C.$$

This is exactly what an optimization system needs. Composing two tunable stages should compose their parameter spaces without manually creating a new mega-config type for every pipeline combination.

4.2 Parameter products are not flat dictionaries

A composed parameter object retains structure:

$$P = P_{chunk} \times P_{repr} \times P_{embed} \times P_{retrieve} \times P_{answer}.$$

An intervention focuses into one subobject through an optic. The dependency graph then determines which downstream material depends on that focus.

This is superior to an untyped key-value parameter registry because structure supports lawful composition, static validation, and localized invalidation.

4.3 Reparameterization

Often a product exposes a simpler parameterization than an internal stage. A reparameterization map

$$r : R \to P$$

induces a new parameterized system

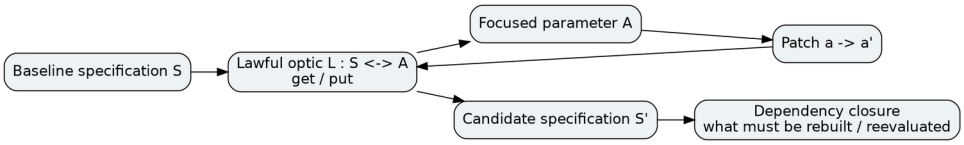
$$f \circ (r \times id_A) : R \times A \to B.$$

This is how product-facing plugins can wrap lower-level tuning knobs. A RAG product might expose quality_profile = balanced while internally mapping it to channel depths, rerank pool, and context budget. The mapping is itself material optimization semantics and should be identified and testable.

5. Optics as intervention interfaces

5.1 Why lenses belong in an optimization system

Optimization is mostly local mutation of immutable global specifications. Lenses describe exactly that operation and carry laws the plugin can test.



A lawful intervention optic focuses one parameter while preserving the surrounding specification.

A plugin can register intervention foci such as:

rag.chunking.max_tokens
rag.embedding.model
rag.retrieve.vector_weight
rag.reranker.pool
rag.answer.context_budget
serve.deadline.rerank

The core does not understand what these values mean. It understands that each optic identifies a lawful projection and update over a schema.

5.2 Beyond simple lenses

Not all intervention relationships are simple product fields.

- A **prism** is appropriate when a candidate selects one alternative in a sum type, such as exact versus ANN backend.

- A **traversal** can target a set of homogeneous route entries.
- A **partial optic** can express a focus valid only under a capability predicate.
- A **bidirectional adapter** can expose a normalized tuning representation while preserving a richer product specification.

The minimal implementation starts with lenses because they cover many practical cases and their laws are easy to test. The plugin protocol should allow richer optic kinds later without changing campaign semantics.

5.3 Interventions carry semantic classes

A patch is more than (optic, value). It should also declare a semantic class and target dependency nodes. For example:

```
type Patch struct {
  ID      string
  Optic   string
  Value   Envelope
  Classes []SemanticClass
  Targets []string
  Closure []string
}
```

The closure should normally be computed rather than trusted. The declaration can be checked against the dependency graph. This makes optimization causally explicit.

Part II. Probability and empirical semantics

6. Markov kernels for stochastic systems

6.1 Evaluation is not a function to a number

Provider-backed systems are stochastic. Even deterministic code may face variable networks, scheduling, caches, or dynamic external services. The semantic type is therefore a probability kernel

$$K : X \rightsquigarrow Y,$$

which assigns each $x \in X$ a probability distribution over Y.

In a finite implementation, one can model this as

```
type Dist[T comparable] map[T]float64
```

and compose distributions with Bind.

The sandbox includes this finite form to make the Kleisli laws executable. Production systems rarely enumerate distributions; they draw samples and retain their material observations.

6.2 Kleisli composition

Given

$$K : X \rightsquigarrow Y , L : Y \rightsquigarrow Z ,$$

the composite is

$$(L \odot K)(x)(z) = \int_Y L(y)(z) K(x)(d\, y).$$

This is the mathematical meaning of composing stochastic stages such as query rewriting, generation, and judging. It explains why preserving one seed at the top of a run is not sufficient: independent or intentionally coupled random choices should be split by stable semantic labels.

6.3 Markov-category viewpoint

A Markov category gives a categorical language for probabilistic processes with copying and discarding of classical data. The architecture does not require a full abstract Markov-category library, but the viewpoint is useful because it clarifies which wiring operations are legitimate for stochastic computations.

Copying an observed value is harmless. Copying a stochastic morphism means sampling twice, not magically reusing one draw. Reusing one draw is a different coupling. An experiment system must make this distinction explicit.

7. Couplings and paired evaluation

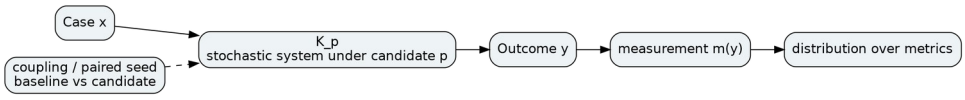
7.1 Marginals are not enough

Suppose the baseline produces $Y_b \sim K_b(x)$ and candidate produces $Y_c \sim K_c(x)$. To estimate their difference efficiently, the experiment needs a joint distribution

$$\gamma_x \in \Gamma\left(K_b(x), K_c(x)\right),$$

whose marginals are the two arm distributions.

This joint distribution is a **coupling**.



Paired evaluation is a coupling between baseline and candidate stochastic kernels.

Using the same case and repeat coordinate is already a coupling at the workload level. Using common random numbers or stable split seeds creates a stronger coupling when providers permit it.

7.2 Exact coordinates

The practical coordinate

$$(i, r, a)$$

for case i, repeat r, and arm a should be immutable. A paired comparison is valid only when both arms have the same case and repeat coordinates. Missing failures must not be silently dropped because that changes the joint sample.

This gives mathematical backing to the strict pairing already present in ragopt.

7.3 Seed splitting

A deterministic root seed can be split by semantic label:

$$s_{child} = H(s_{parent}, \text{label}).$$

The sandbox uses this approach. A larger system can split by campaign, candidate, case, repeat, operation, and provider stage. This provides reproducibility without assuming every stochastic provider accepts explicit seeds.

When a provider does not expose seed control, the retained output artifact becomes the material sample. Replay can then hold provider output fixed while testing deterministic downstream changes.

8. Measurements form a structured object

8.1 Rejecting the universal scalar objective

Optimization libraries often expect

$$f : S \to R.$$

RAG optimization instead yields a vector such as

$$(recall, nDCG, grounding, failures, latency, tokens, dollars, disclosure).$$

Some dimensions are maximize, some minimize, some are hard constraints, and some are meaningful only within strata. There is no canonical natural transformation from this measurement object to one real number.

A plugin should therefore register metric definitions and decision relations separately.

8.2 Measurement monoids

Many operational measurements combine monoidally:

- counts by addition;
- cost by addition;
- maximum latency by max;
- histograms by binwise addition;
- sets of violations by union;
- traces by concatenation.

This structure enables compositional observation of tensor and sequential plans. It also explains why static and dynamic analyses can share a common fold pattern.

8.3 Ordered gates

A decision policy is better represented as an ordered composition of predicates:

$$G = G_{security}; G_{integrity}; G_{coverage}; G_{quality}; G_{cost}.$$

Later metrics are relevant only after earlier hard constraints pass. This is lexicographic or staged decision semantics, not a weighted sum.

The optimizer may still rank surviving candidates on a preference metric or Pareto frontier. But feasibility and preference remain distinct concepts.

Part III. Campaigns as feedback systems

9. Coalgebraic view of optimization

9.1 Campaign state

Optimization is an iterative process. Let campaign state be

$$C = (s, E, B, H),$$

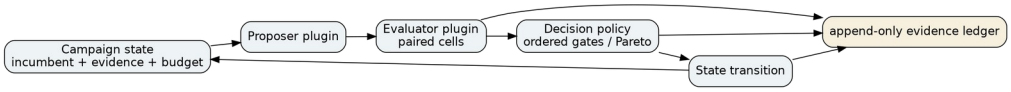
where s is the current incumbent, E accumulated evidence, B remaining budget, and H campaign history.

A campaign step observes state and produces an action plus new state:

$$\delta : C \to F(C),$$

for an appropriate functor F describing proposal, evaluation requests, decision, stop, or external review.

This coalgebraic framing is useful because optimization is a process that can be resumed from observations. It is not merely a pure function from a candidate list to a winner.



A campaign is a feedback process over immutable accumulated evidence.

9.2 Proposer is a plugin, not the kernel

The proposer may be:

- exhaustive enumeration;
- random sampling;
- Bayesian optimization;

- evolutionary mutation;
- gradient-based update;
- human proposal;
- LLM-generated candidate;
- a domain-specific heuristic.

The core requires only that proposals are materialized as lawful interventions with identities. This prevents a fashionable search strategy from becoming the architecture.

9.3 Evaluator as an interpreter

The evaluator takes a candidate specification and case, realizes required artifacts, runs the relevant system interpreter, and projects metrics. It is domain-owned.

In RAG, one evaluator may stop at retrieval. Another may run complete answers. A third may execute multi-turn sessions. The campaign core sees typed trials and evidence references.

9.4 Decision as a relation

Selection is a relation over evidence, not necessarily an argmax. A decision procedure can return:

- candidate preferred;
- baseline preferred;
- incomparable;
- insufficient evidence;
- invalid candidate;
- requires human review.

This is particularly important for multi-objective and safety-constrained optimization.

10. Evidence and provenance

10.1 Evidence is not logging

Every campaign fact needed for a decision should be materialized:

- baseline specification identity;
- intervention identity;
- realized candidate specification;
- workload and case identity;
- stochastic coordinate;
- trial outcome including failure;
- native artifact references;
- metric projection;
- decision gates;
- final promotion recommendation.

An append-only event ledger is a simple implementation. The sandbox writes trial and decision events to JSONL and resumes completed coordinates on the next run.

10.2 Provenance semiring intuition

Database provenance asks how outputs depend on inputs. Optimization requires a similar relation: which evidence supports a decision, and which source artifacts produced that evidence?

One can view provenance annotations as living in a semiring-like algebra where alternative derivations combine additively and jointly required facts combine multiplicatively. The implementation need not expose symbolic polynomials, but the principle suggests a concrete API: every derived artifact and metric references its parent identities.

10.3 Reproducibility boundary

A reproducible experiment is not necessarily one that regenerates identical model text from scratch. It is one whose material inputs and sampled outputs are retained sufficiently to replay the deterministic comparison. The architecture distinguishes:

- semantic reproducibility: same declared stochastic process;
- material reproducibility: same retained samples/artifacts;
- operational reproducibility: same infrastructure trace, usually neither required nor desirable.

Part IV. Plugin architecture

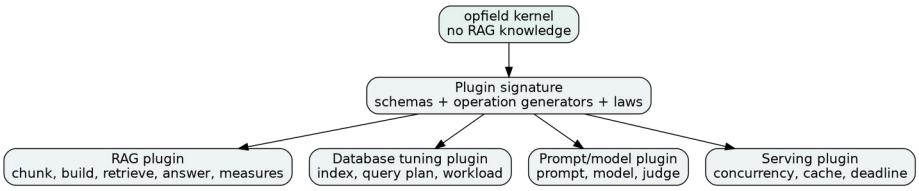
11. The plugin signature

11.1 What a plugin may contribute

A domain plugin should be able to register:

1. schemas and codecs;
2. primitive operation descriptors;
3. executors for those primitives;
4. laws and conformance tests;
5. intervention optics;
6. dependency nodes/edges;
7. workload adapters;
8. evaluators and native artifact schemas;
9. metric definitions;
10. optional decision gates.

It should **not** receive unrestricted access to campaign storage, global mutation, or hidden orchestration internals.



Domain plugins graft their signatures onto a domain-neutral kernel.

11.2 Why registration beats inheritance

Go interfaces are well suited to capability boundaries. A plugin installs operations into a registry. The registry validates unique operation IDs and executes plugin laws during registration. This keeps dependencies one-way: the kernel knows only interfaces; the plugin imports the kernel.

An inheritance-style framework would encourage every domain to subclass one enormous experiment type. Registration keeps features orthogonal and versionable.

11.3 Primitive operation contract

A primitive operation descriptor should answer questions before execution:

- what input/output schemas exist;
- what effects are possible;
- which semantic dependency nodes it reads;
- whether it is deterministic;
- whether results may be cached by semantic identity;
- rough resource/cost hints;
- plugin and operation version.

The executor then supplies the dynamic implementation.

This is the seam where providers and external systems enter. A reranker operation can be declared as network effect, non-deterministic, cost-bearing, and dependent on query.reranker.model. A local RRF operation is pure, deterministic, and cheaply replayable.

11.4 Plugin laws

A plugin should be rejected if its declared laws fail. Examples include:

- lens laws;
- canonical codec round-trip;
- total ranking order;
- deterministic output under declared deterministic operation;
- no unauthorized output under a filter primitive;
- incremental/full-build equivalence for an index plugin;
- measure identity and aggregation laws.

This makes plugin registration analogous to supplying both an implementation and evidence that it respects the core algebra.

12. Free plans as the stable extension point

12.1 Plugins provide generators, applications provide wiring

The most important interface decision is that plugins register **generators**, not complete hidden pipelines. Product code can then wire generators into plans. This lets the core inspect and transform the plan.

For a RAG route:

$$Q \xrightarrow{rewrite} Q' \xrightarrow{retrieve_L \otimes retrieve_V} R_L \otimes R_V \xrightarrow{fuse} R \xrightarrow{rerank} R' \xrightarrow{admit} E.$$

Each primitive comes from one or more plugins. The plan syntax remains shared.

12.2 Multiple interpretations

The same RAG query plan can be interpreted as:

- production execution;
- local deterministic simulation;
- static data-disclosure analysis;
- provider-cost estimate;
- dependency closure;
- cache-key construction;
- Graphviz diagram;
- experiment trace schema.

This is the strongest argument for a free plan representation. A callback graph usually bakes execution into its nodes and leaves every other interpretation to ad hoc reflection.

12.3 Controlled escape hatches

Not every system can be decomposed conveniently. The architecture can permit an opaque primitive that encapsulates a complex service. But opacity has a cost: static analysis stops at that boundary. The descriptor must conservatively declare effects, dependencies, cost, and semantic identity.

This provides a pressure gradient toward useful decomposition without making decomposition mandatory.

Part V. RAG as a grafted optimization field

13. RAG parameterization

13.1 A complete RAG specification

A useful RAG optimization specification can be modeled as a product:

$$S_{RAG} = S_C \times S_N \times S_K \times S_R \times S_E \times S_I \times S_Q \times S_A \times S_O,$$

where the factors denote corpus/admission, normalization, chunking, representations, embeddings, index, query, answer/agent policy, and serving operations.

Not every campaign exposes every factor. An optic focuses the subset under study.

13.2 Dependency graph

The graph carries causal invalidation:

```
corpus
  -> normalize
  -> chunk
  -> representation
  -> embedding
  -> vector-index

chunk -> lexical-index
query-policy -> retrieval
lexical-index -> retrieval
vector-index -> retrieval
retrieval -> answer
answer -> session/frontend evaluation
```

A candidate changing query.vector_weight does not rebuild an index. A candidate changing chunking invalidates nearly everything downstream. The field knows this before running the candidate.

13.3 Optimization fidelities

RAG naturally supplies multiple evaluators:

1. build/integrity laws;
2. exact retrieval evaluation;
3. approximate-index oracle comparison;
4. answer evaluation;
5. multi-turn tool/session evaluation;
6. shadow production trace;
7. canary online metrics.

A semantic class determines the minimum required fidelity. The generic campaign kernel can schedule these stages without knowing what nDCG or grounding means.

14. Worked RAG plugin architecture

14.1 Indexing plugins

An indexing plugin may register generators:

```
normalize : SourceRevision -> Document
chunk     : Spec x Document -> ChunkSet
represent : RepSpec x ChunkSet -> RepresentationSet
embed     : EmbedSpec x RepresentationSet -> VectorSet
lexical   : LexicalSpec x ChunkSet -> LexicalIndex
vector    : VectorSpec x VectorSet -> VectorIndex
```

It can also register optimizer foci such as chunk size, overlap, representation kinds, embedding model, ANN search effort, and index parameters.

The core sees schemas, operations, effects, dependencies, and laws. RAG-specific artifacts remain in the plugin.

14.2 Query plugins

A query plugin registers rewrite, channels, collapse, fusion, filtering, reranking, context admission, and answer primitives. A product may add connected retrieval or structured facts as additional generators.

Security policy can be checked statically when descriptors state which operations disclose text remotely and which produce authorization certificates.

14.3 Evaluation plugins

Retrieval evaluation produces native data such as ranks and judgments. Answer evaluation may invoke a judge. Session evaluation may drive a live product runtime. Each evaluator implements the same trial interface while retaining its native artifact.

The generic trial contains only comparable projections and coordinate identity.

15. How this maps to ragopt

15.1 What should remain

The strongest parts of ragopt should remain conceptually unchanged:

- immutable candidate inputs;
- exact case/repeat/arm coordinates;
- append-only cell custody;
- resumability;
- strict comparison;
- ordered gates;
- promotion reports that do not deploy directly.

The mathematical work strengthens the layer beneath candidate creation and evaluator execution.

15.2 Proposed package split

A practical design is:

```
opfield/ or optimize/kernel/
  core/       schemas, descriptors, effects, identities
  plan/       free typed process syntax and folds
  plugin/     registries and law contracts
  optic/      intervention interfaces
  prob/       stochastic coordinates and coupling contracts
  evidence/   artifacts and provenance

ragopt/
  candidate/
  eval/
  compare/
  gate/
  runstore/
  report/

ragopt/ragspace/
  RAG parameter/dependency/fidelity adapters
```

Whether the domain-neutral layer lives in ragopt or a separate small module is an organizational decision. The architectural rule is more important: ragopt must not import RAG concepts into its core.

15.3 Candidate creation through optics

Instead of copying a directory and mutating a named file as the deepest abstraction, a candidate can be defined first as:

```
baseline specification ID
+ optic ID
+ typed new value
+ intervention class
+ computed dependency closure
```

Materialization to files is an interpreter. This retains compatibility with pragmatic applications while making candidate semantics explicit.

15.4 Arm as a field interpreter

An existing ragopt arm becomes one evaluator of the field. It receives the realized candidate and exact coordinate, runs the product-native program, writes its native artifact, and returns common measurements. This is already close to how the RAG-TTC adapter works.

Part VI. The executable sandbox

16. Sandbox architecture

16.1 Goals

The sandbox is deliberately small enough to read in one sitting. It demonstrates five claims:

1. plugins can remain ordinary Go interfaces;
2. typed plans can be explicit data rather than opaque closures;
3. the same plan supports execution and static analysis;
4. intervention laws can be executable;
5. a domain-specific RAG optimizer can graft onto the core without introducing RAG types into campaign infrastructure.

16.2 Package map

core/ canonical identities, schemas, ports, effects, execution outcomes
plugin/ plugin and operation registry
plan/ free plan syntax, validation, static analysis, identity
engine/ plan execution
optic/ lawful lenses
prob/ finite distributions and deterministic seeds
experiment/ intervention spaces, cases, coordinates, runners
campaign/ append-only resumable campaign engine
domain/ragtoy/
miniature RAG plugin and optimization space
cmd/opfield-demo/
runnable demonstration

16.3 The toy RAG domain

The toy domain contains four gardening documents and three queries. Retrieval blends exact lexical overlap with a tiny deterministic semantic concept mapping. The optimization space varies semantic weight through a typed intervention. The runner reports MRR and a utility containing a small complexity penalty.

This model is intentionally too small to be a useful retriever. Its purpose is architectural: the campaign engine has no idea what a document, query, MRR, or semantic weight is.

16.4 Resumability

Each trial coordinate is written to an append-only JSONL event store. Re-running the demo reads completed coordinates and schedules only missing cells. This demonstrates the coalgebraic/evidence view in a minimal implementation.

The run command is:

```
go test ./...
go run ./cmd/opfield-demo -out ./demo-out
```

The generated plan.json contains operation topology, effects, dependencies, cost hints, and semantic plan identity. events.jsonl contains immutable trial events. result.json contains the selected candidate and aggregate utility projections.

17. Correctness laws implemented in the sandbox

17.1 Envelope identity

Typed values are serialized into envelopes with a domain-separated digest. Decode verifies the digest before use. This demonstrates material identity without requiring a full artifact store.

17.2 Plan typing

Sequential composition validates equality of adjacent ports. Primitive plans are checked against registered descriptors. Copy, drop, and permutation carry explicit typed ports.

17.3 Plan analysis

Static analysis walks the same plan syntax used by execution and accumulates operations, effects, semantic dependencies, deterministic/cacheable flags, and cost estimates.

17.4 Lens laws

The RAG plugin registers a law for its semantic-weight lens. Plugin registration fails if the law fails on supplied representative values.

17.5 Probability law

The test suite checks associativity of finite-distribution Bind, demonstrating the Kleisli composition law in an executable finite setting.

Part VII. Design boundaries and research program

18. What belongs in the kernel

The kernel should contain only structures that are stable across domains:

- typed immutable values and identities;
- primitive operation descriptors;
- free plan composition;
- effect/dependency/cost annotation vocabulary;
- plugin registration and laws;
- generic intervention interfaces;
- stochastic coordinates/coupling hooks;

- trial/evidence coordinates;
- campaign state and resumable custody.

It should not contain retrieval metrics, LLM providers, chunkers, SQL tuning knobs, prompt formats, or UI concepts.

A useful test is: could a database-query optimizer use the abstraction without pretending to be RAG? If not, it probably belongs in the RAG graft.

19. What belongs in plugins

Plugins own semantic vocabulary that varies by domain:

- schemas;
- primitive operation meaning;
- codecs;
- capability constraints;
- domain laws;
- parameter optics;
- dependency edges;
- native artifacts;
- workload semantics;
- metrics and protected strata;
- domain-specific decision gates.

Plugins may depend on provider libraries and product packages. The core must not.

20. What remains outside both

Some responsibilities should remain external even to domain plugins:

- authentication and deployment authority;
- production scheduler choice;
- secrets management;
- organization-specific approval workflow;
- user-facing UI;
- long-term artifact retention policy;
- provider procurement or legal policy.

The optimization system can emit evidence consumed by these systems without taking ownership of them.

21. Research directions

21.1 Richer optics

A future kernel can add prisms, traversals, and profunctor optics for sum types and collections. The important constraint is that the resulting intervention laws remain executable and identity-preserving.

21.2 Open systems and wiring diagrams

RAG services interact with providers, stores, and frontend runtimes. Structured or decorated cospans and open-system categories may provide a useful representation for compositional systems with explicit interfaces and shared resources. This is most promising for modeling deployment topology and resource sharing rather than ordinary pure dataflow.

21.3 Double categories

Optimization has two kinds of composition: systems compose horizontally, while transformations/interventions between systems compose vertically. Double categories or equipments may provide a natural setting for this two-dimensional structure. A practical architecture should wait until concrete API pressure justifies exposing it.

21.4 Reverse differentiation

Gradient-based learning can be understood categorically through reverse derivative categories, lenses, and functorial accounts of backpropagation. This could permit gradient-based proposers to live inside the same field abstraction as black-box search. The field supplies the parameterized semantics; a differentiable plugin supplies reverse structure. Non-differentiable RAG stages simply do not implement that capability.

21.5 Bayesian decision plugins

The decision relation can be generalized from frequentist paired summaries to posterior decision rules. A Bayesian plugin might produce distributions over metric differences and compute probability of constraint satisfaction. The campaign evidence contract need not change.

21.6 Adaptive experiment allocation

A proposer/allocator can use evidence state to decide which candidate-case pair to evaluate next. This introduces selection bias that must be retained in evidence. Coalgebraic campaign semantics are well suited to adaptive allocation because the allocation decision is itself an event.

22. Architectural recommendations

The immediate recommendations for the RAG program are:

1. Keep ragopt's experiment custody semantics.
2. Introduce a small domain-neutral typed operation/plan layer rather than expanding ragopt with RAG-specific concepts.

3. Make candidate interventions typed optics over immutable specifications.
4. Compute invalidation through dependency graphs.
5. Treat stochastic evaluation through exact coordinates and explicit coupling/seed policy.
6. Preserve native product artifacts; project only comparable metrics into the generic layer.
7. Let plugins register generators and laws, not arbitrary orchestration engines.
8. Interpret the same free plan into execution, static analysis, identity, diagrams, and tests.
9. Keep decision gates separate from proposer/search strategy.
10. Require every new abstraction to demonstrate at least two distinct domain grafts before enlarging the core.

23. Conclusion

Optimization architecture becomes simpler when the word *optimizer* is moved upward rather than downward. The stable substrate is not a search algorithm. It is a mathematical field of composable systems, local interventions, stochastic observations, structured measurements, evidence, and feedback.

The categorical structures are useful because they identify the right seams. Free monoidal syntax separates plugin generators from orchestration. Parameterized categories explain how tunable systems compose. Optics give lawful local interventions. Markov/Kleisli structure explains stochastic evaluation. Couplings explain strict paired comparison. Coalgebra explains resumable iterative campaigns. Provenance explains why decisions must retain derivation evidence.

The resulting software core can remain small. The accompanying Go implementation is intentionally modest: no external libraries, no universal workflow language, no reflection-heavy dependency injection, and no RAG types in the campaign package. Yet it supports typed composition, static analysis, plugin laws, stochastic coordinates, interventions, resumability, and a working RAG optimization graft.

That is the architectural target: **strong semantics in a small kernel, with domain richness supplied through constrained plugins rather than absorbed into the core.**

Appendix A. Core interfaces

A production version can evolve toward interfaces like:

```
type Plugin interface {
   Manifest() Manifest
   Install(*Builder) error
   Laws() []Law
}

type Operation interface {
   Descriptor() Descriptor
   Execute(context.Context, Frame) Execution
}
```

```
type Optic interface {
    ID() OpticID
    SourceSchema() SchemaID
    FocusSchema() SchemaID
    Get(Envelope) (Envelope, error)
    Put(Envelope, Envelope) (Envelope, error)
    Laws() []Law
}

type Space interface {
    ID() string
    Baseline(context.Context) (Envelope, error)
    Proposals(context.Context, Envelope, EvidenceView) ([]Patch, error)
    Apply(context.Context, Envelope, Patch) (Envelope, error)
}

type Runner interface {
    ID() string
    Run(context.Context, TrialRequest) TrialResult
}

type DecisionPolicy interface {
    Decide(context.Context, ComparisonSet) Decision
}
```

The interfaces are intentionally narrow. Rich data lives in typed envelopes and immutable artifacts rather than growing the method surface.

Appendix B. Category-to-code correspondence

Mathematical structure	Software representation
Object	schema/port type
Primitive morphism	registered operation
Identity	identity plan
Composition	sequence plan
Tensor product	parallel/tensor plan
Symmetry	permutation plan
Diagonal	immutable copy plan
Terminal/discard	drop plan
Free category	plan syntax generated from operations
Algebra/interpreter	executor, analyzer, identity fold, renderer
Parameter object	immutable specification component
Para morphism	tunable operation family
Lens/optic	intervention focus
Markov kernel	stochastic evaluator/provider stage

Mathematical structure	Software representation
Kleisli composition	stochastic stage composition
Coupling	paired arm execution/seed policy
Measurement	typed metric projection
Coalgebra	campaign feedback transition
Provenance	evidence/artifact dependency graph

Appendix C. Suggested RAG plugin split

A realistic RAG implementation should prefer several small plugins over one monolith:

rag-corpus-plugin
rag-chunking-plugin
rag-representation-plugin
rag-embedding-plugin
rag-lexical-plugin
rag-vector-plugin
rag-retrieval-plugin
rag-rerank-plugin
rag-answer-plugin
rag-agent-plugin
rag-serving-plugin
rag-eval-plugin

Product repositories then add:

gec-policy-plugin
ttc-product-plugin
garden-presentation-plugin

A plugin package may register multiple operations where they form one coherent capability. The split is conceptual, not a mandate for one repository per item.

Appendix D. Sandbox verification

The delivered sandbox was verified with:

```
go test ./...
go run ./cmd/opfield-demo -out /mnt/data/opfield_demo
```

The test suite checks envelope integrity, plan composition, plugin laws, and finite-distribution bind associativity. The demonstration evaluates five specifications over three RAG cases with three repeats each, producing 45 exact trial coordinates. The campaign writes append-only events and chooses a winner from the measured utility. Re-running against the same event store resumes rather than duplicating trial coordinates.

Appendix E. Bibliography

Abramsky, Samson, and Bob Coecke. 2004. “A Categorical Semantics of Quantum Protocols.” Proceedings of LICS.

Baez, John C., and Jason Erbele. 2015. “Categories in Control.” Theory and Applications of Categories 30.

Claessen, Koen, and John Hughes. 2000. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” ICFP.

Fong, Brendan, David I. Spivak, and Rémy Tuyéras. 2019. “Backprop as Functor: A Compositional Perspective on Supervised Learning.” LICS.

Fritz, Tobias. 2020. “A Synthetic Approach to Markov Kernels, Conditional Independence and theorems on Sufficient Statistics.” Advances in Mathematics 370.

Gavranović, Bruno. 2020-2024. Work on categorical cybernetics, parameterized categories, and compositional machine learning.

Green, Todd J., Gregory Karvounarakis, and Val Tannen. 2007. “Provenance Semirings.” PODS.

Kock, Anders. 1972. “Strong Functors and Monoidal Monads.” Archiv der Mathematik.

Lawvere, F. William. 1963. “Functorial Semantics of Algebraic Theories.” Proceedings of the National Academy of Sciences.

Mac Lane, Saunders. 1971. *Categories for the Working Mathematician*. Springer.

Moggi, Eugenio. 1991. “Notions of Computation and Monads.” Information and Computation 93(1).

Riley, Mitchell. 2018. “Categories of Optics.” arXiv preprint.

Shapiro, Marc, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. “Conflict-Free Replicated Data Types.” SSS.

Spivak, David I. 2014. *Category Theory for the Sciences*. MIT Press.